



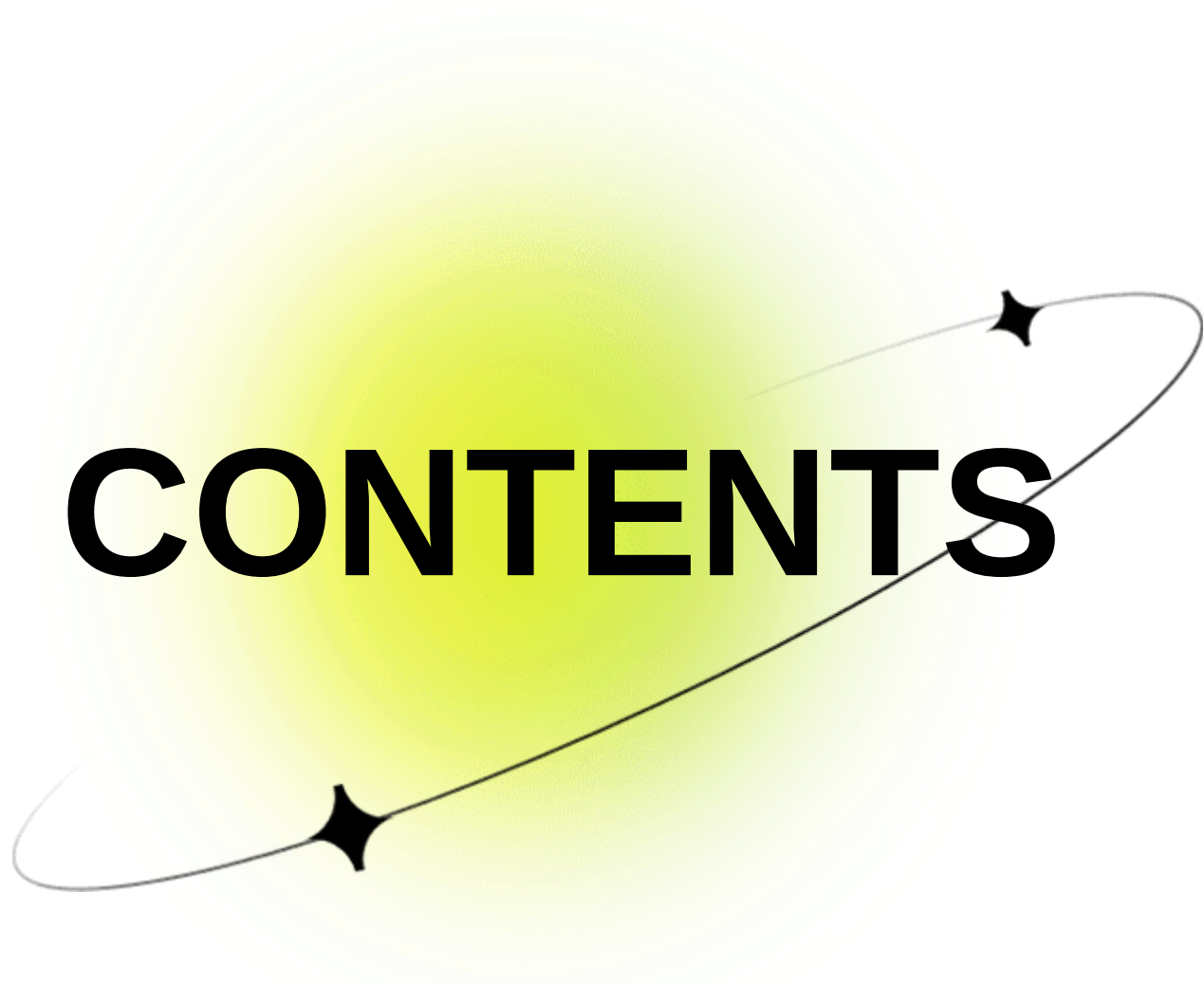
CMR COLLEGE OF ENGINEERING & TECHNOLOGY
AUTONOMUS INSTITUTION WITH NAAC ACCREDITATION (A+ GRADE)
APPROVED BY AICTE, PERMANENTLY AFFILIATED TO JNTUH, NBA ACCREDITATION
KANDLAKOYA (V), MEDCHAL ROAD, HYDERABAD -501401

Emotion2Emoji: DeepFace Magic

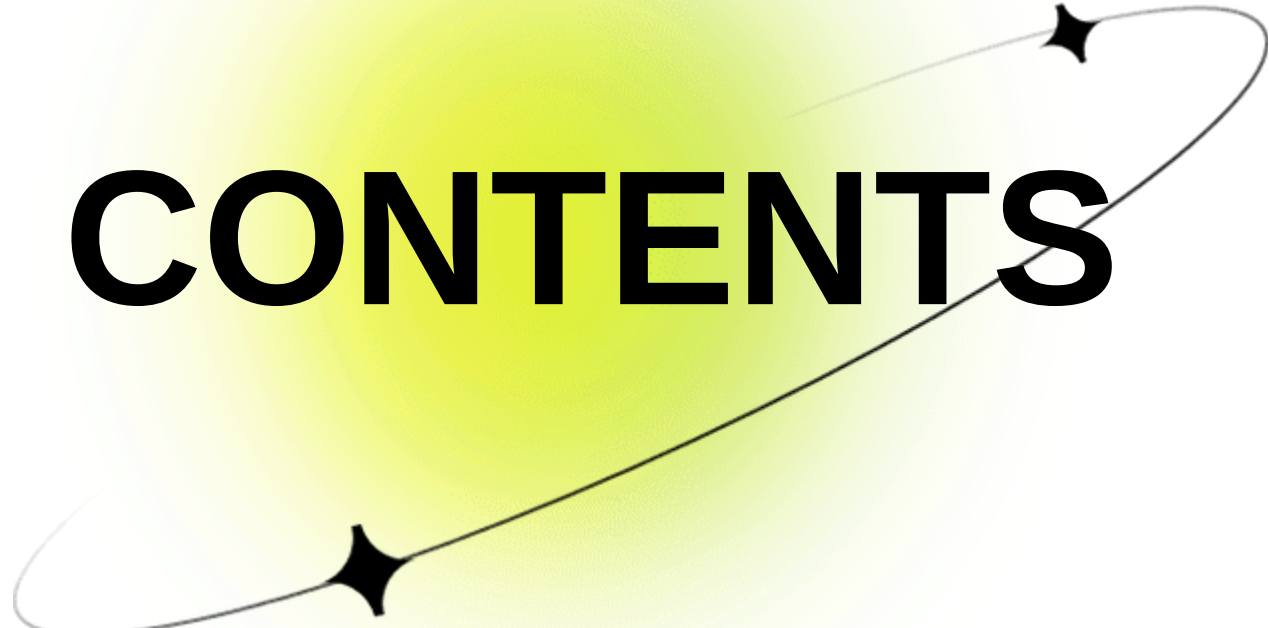
Krishna - 23H51A66H2

Nandhini - 23H51A66H8

MedhaSri - 23H51A66J3



CONTENTS



01 | Why Emotion2Emoji?

02 | DeepFace Toolkit

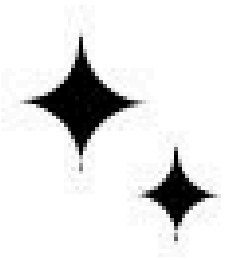
03 | Pipeline Overview

04 | Code Walkthrough



01

Why Emotion2Emoji?

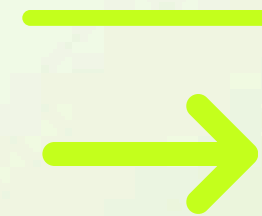


From Face to Emoji in One Click

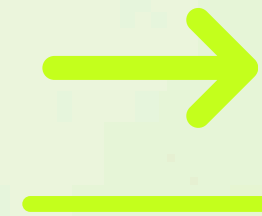
Transform any portrait into the perfect emoji within seconds, bridging human expression and digital reaction effortlessly.



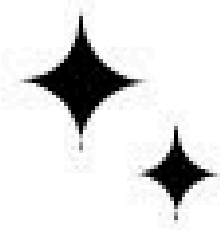
Human Expression



DeepFace Magic



Digital Reaction



Market & Chat App Demand



Booming Emoji Usage

Over 10 billion emojis are sent daily, with a rising demand for more personalized and authentic digital expression.



Current Gaps

Existing keyboard suggestions often misinterpret user intent, leading to a gap for automatic, authentic emoji creation that truly reflects user emotion.

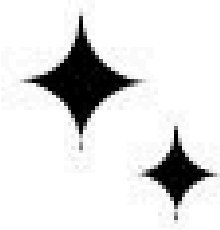


The Opportunity

A solution like Emotion2Emoji can significantly increase user engagement for social platforms by making digital communication more nuanced and personal.

02

Pipeline Overview



End-to-End Pipeline Flow



1. Image Ingestion



2. Face Detection

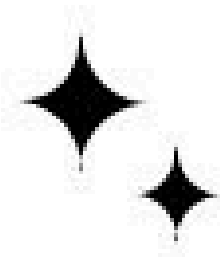


3. Emotion Classification



4. Emoji Mapping

Each stage is modular, allowing for easy customization and integration of regional emoji packs.

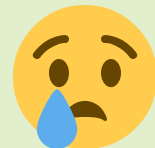


Emotion-to-Emoji Mapping

A precise lookup table ensures users worldwide receive meaningful and culturally relevant emojis, with smart fallbacks for close scores.



Happy



Sad



Surprised



Angry



Disgust



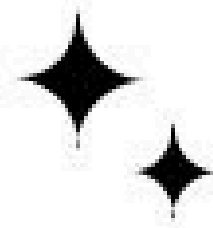
Fear



Neutral

03

DeepFace Toolkit

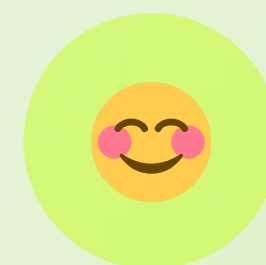


What DeepFace Delivers

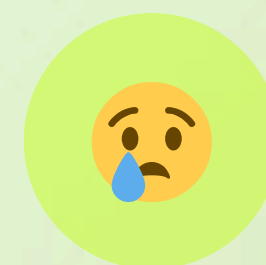
A lightweight, TensorFlow-based library that analyzes faces and outputs seven key emotion probabilities.

- ✓ Real-time capability on CPU
- ✓ Open-source license
- ✓ Ideal for commercial chat plugins

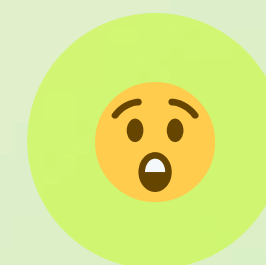
Detected Emotions



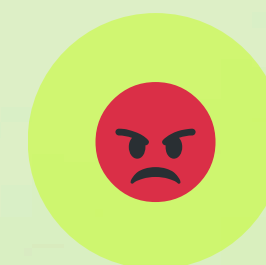
Happy



Sad



Surprised



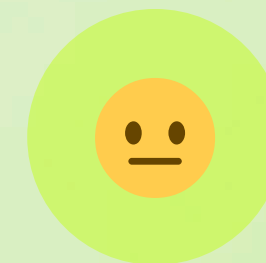
Angry



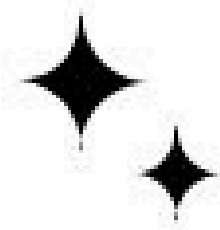
Disgust



Fear



Neutral



Lightweight vs. Heavy Models

MobileNet-V2 (Our Choice)

- ✍ Size: ~14 MB
- ⚡ Speed: ~50ms on CPU
- 🎯 Accuracy: ~90%

Perfect for mobile keyboards without sacrificing UX or battery life.

Larger Alternatives

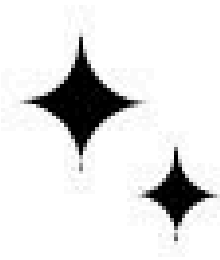
- 📦 Size: > 100 MB
- ⌚ Speed: Slower, often needs GPU
- 🎯 Accuracy: ~92-95%

Overkill for simple emoji mapping, resource-intensive.



04

Code Walkthrough



Code: Setup & User Input

The code begins with robust helper functions to filter image files and safely gather user input via the command line.

- ▼ **list_images():** Filters for common formats like .jpg, .png, and .bmp.
- **CLI Interaction:** Guides the user to select a folder and choose an image from a numbered list.
- 🛡️ **Defensive Coding:** Includes checks for invalid paths, non-image files, and out-of-range choices to prevent crashes.

```
# Helper function to list image files
def list_images(folder="."):
    exts = (".jpg", ".jpeg", ".png", ".bmp")

    return [f for f in os.listdir(folder) if f.lower().endswith(exts)]

# Main function for user interaction
def main():
    folder = input("📁 Enter folder path: ").strip() or "."
    images = list_images(folder)
    ... # Display choices and get input
```

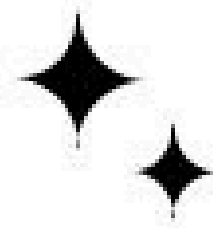


```
# Core function to analyze emotion
def detect_emotion(image_path):
    try:
        result = DeepFace.analyze(
            img_path=image_path,
            actions=["emotion"],
            enforce_detection=True
        )
        dominant = result[0]["dominant_emotion"]
        print(f"Dominant Emotion: {dominant}")
    except Exception as e:
        print(f"Error: {e}")
```

Code: Emotion Detection

The heart of the script is a single call to the DeepFace library, which handles the complex analysis.

- ⚙️ **DeepFace.analyze():** The core function that performs emotion analysis.
- 🔒 **enforce_detection=True:** Ensures a face is present, preventing errors on unsuitable images.
- ⚠️ **Exception Handling:** Gracefully manages errors for low-resolution or occluded faces, ensuring the application remains stable.



Code: From Print to Return

A simple modification transforms the script from a standalone tool into a reusable function for apps and APIs.

Original: Print to Console

```
dominant = result[0]["dominant_emotion"]  
print(f"Dominant Emotion: {dominant}")
```



Modified: Return Emoji

```
dominant = result[0]["dominant_emotion"]  
emoji = emotion_to_emoji_map[dominant]  
return emoji
```

This change makes the function callable by web APIs or mobile apps, returning the emoji string directly.

THANK YOU

2025/01/01